

Do **not** prompt the model with security-sensitive work of any kind including security and intrusion, credentials and secrets, surveillance and privacy, model safeguards, NSFW content, fraud, and other restricted conduct. These are grounds for immediate removal from the project.

Prompt Classification

Your goal is to write a prompt that clearly falls into one of the categories listed below. Do not explicitly mention or name the task category in the prompt itself, it should be inferable from the content and framing of the turn, not stated outright.

A full submission will typically span multiple turns, with each turn targeting a different category. For example, Turn 1 might focus on root cause analysis, Turn 2 on code review, and so on. When you submit the completed task, you'll be asked to specify the classification for each individual turn.

*** Currently the highest-need categories:** Root Cause Analysis, System-Level Investigation, and Maintenance & Ops Tooling. Prioritize these.

Category	What it means	Cause of Error in Classification
Debugging	Finding and fixing a specific broken behavior described in the prompt. A fix and nothing more.	Picked when regression prevention is also explicitly requested (that is RCA).
Root Cause Analysis *	Fixing a broken behavior PLUS explicit regression prevention (a test or guard so it cannot recur). Both parts must be in the prompt.	Picked when only a fix is asked for. A careful fix alone is still Debugging.
System-Level Investigation *	Fixing a problem whose root cause is a mismatch between the environment and the code (versions, config, deployment differences), not a plain code bug (e.g., fixing problems caused by mismatches between the environment and the code).	Focusing on the bug fix instead of checking whether the root cause is a mismatch between the environment and the code.
Code writing (Feature Implementation, Refactor)	Adding, changing, restructuring, or hardening behavior in code that is not broken, (e.g., feature implementation, refactoring and cleanup, scripts and automation, version-control surgery).	Used as a catch-all for any prompt that produces code. Check the prompt direction to ensure this is not being inaccurately overused.
Code review	Evaluating or critiquing existing code, not writing it. (e.g., implementation-level review, architecture/system-level review).	Review asks that focus on environment or system fit belong to System-Level Investigation.
Exploration & learning	Understanding existing code or behavior, with no change requested; the model explains, (e.g., "How does this subsystem work," "walk me through this function," onboarding to an unfamiliar codebase, learning a new technology.)	Focusing on "understand why this happens" and stopping there, missing that the underlying answer is really a System-Level Investigation or RCA in disguise.
Maintenance & ops tooling *	Observability, performance measurement, or dev-loop tooling; not user-facing behavior, (e.g., Perf measurement, observability/telemetry config, local	Defaulting to Code Writing because these prompts read as concrete build requests. Ultimately missing that the thing being built is internal tooling (observability, performance measurement, dev-loop tooling) rather than

	dev-env iteration, data analysis.)	user-facing behavior, which is the actual distinguishing signal.
Planning & requirements	A scoping or prioritization question with no specific change described yet ("what do we need for launch?", "X or Y first?"), (e.g., scoping discussions, requirement clarification, breaking down a vague ask).	A narrative problem description followed by what to do about it is Code writing, not planning.
Design	A structural or architectural decision as the deliverable itself: schema, API shape, system boundaries. (e.g., System, API, schema, or interface design.)	Confused with Planning & requirements, and feature prompts that merely imply structure get labeled Design.
Testing	Writing tests or interactively verifying behavior as the main task, (e.g., adding a test suite, writing tests for untested code, manual/interactive testing).	Feature prompts that include "and add tests" as a secondary ask get labeled Testing; the primary ask decides.
Deployment & infra	Getting code to run somewhere: CI/CD, cloud, release mechanics. (e.g., cloud platform ops, CI/CD setup, release mechanics).	Labeling these Code Writing because the request involves writing config or scripts, missing that the goal is getting code to run somewhere (CI/CD, cloud, release mechanics), not building application behavior.
Communication	Writing for humans: docs, runbooks, READMEs, design docs, (e.g., technical docs, runbooks, READMEs, design-doc drafting).	Taskers label these Code Writing because the deliverable is text produced alongside a codebase, missing that the actual output is writing for humans (docs, runbooks, READMEs, design docs), not code.

Examples of misclassifications

Code writing vs Maintenance & ops tooling. Both are build requests. If the output serves users of the product, it is Code writing. If it serves the team operating or developing the product (metrics, telemetry, profiling, dev scripts), it is Maintenance & ops tooling.

Code writing vs Communication. If the deliverable is a document a human will read (README, runbook, design doc), it is Communication, even when generated inside a repo alongside code.

Code writing vs Design. Design is when the structural decision IS the deliverable ("how should we shape this API?"). A feature request that implies structure underneath is Code writing.

Code writing vs Testing. Testing only when tests or verification are the primary ask. A feature with tests attached is Code writing.

Code writing vs Deployment & infra. If the ask is about the code's own behavior, Code writing. If it is about getting the code to run somewhere (pipeline, cloud, release), Deployment & infra.

Debugging vs Root Cause Analysis. Both start from something broken. Debugging: fix only. RCA: fix plus an explicitly requested test or guard against recurrence. If the prompt does not ask for prevention, it is Debugging.

System-Level Investigation vs Debugging/RCA. All three start from broken behavior. If the cause is an environment-code mismatch (prod on a different runtime version, config drift, deployment differences), it is System-Level Investigation, not a code-level fix.

System-Level Investigation vs Exploration & learning. Exploration is understanding with nothing broken. If the prompt investigates why something misbehaves across environments, it is System-Level Investigation.

Design vs Planning & requirements. Planning decides WHAT to build (scope, priorities); Design decides

HOW to build it (structure, schema, interfaces). A prompt asking "is X in scope?" is Planning; "how do we structure X?" is Design.

Core principle

A prompt should read like one engineer messaging another, not an instruction to generate text. Don't make prompts sound like forced token burn. Make them sound like real engineering requests that are natural and require work. The model should use many tokens because the task requires real work, not because the prompt artificially demands a long answer.

In long horizon workflows, each prompt should be treated as a step in a continuous engineering process. The exact category may change between turns, discussion, implementation, debugging, testing, refactoring, etc. However the structure of a good prompt should remain. When prompting on taiga the goal is not to describe in minute detail every step, or implementation. Remember the goal of the project is to test SOTA model behavior, across various axes. Ambiguity is useful because it lets the model choose an approach. However it's important to be clear about the outcome, constraints, and the environment.

Anatomy of a good prompt

You do not want to tell the model every exact line to write. It is generally better to describe the specific behavior, leaving the model to find the best implementation. Furthermore a tasker should not try to satisfy the token requirement by asking the model to be endlessly exhaustive. Instead each turn needs to ask the model to do something useful.

A robust prompt should inform the model of:

- **Goal:** the outcome wanted from this turn.
- **Carried context:** what from earlier turns still matters.
- **Scope:** the specific files, behaviors, or requirements in play.
- **Guardrails:** what shouldn't be touched, changed, or assumed.

Questions to consider:

- Would this make sense if sent to a colleague?
- Is the goal clear while the approach stays open?
- Does it follow from what the model just did?
- Do its tokens come from real investigation, not mandated verbosity?
- Have I avoided naming any behavior I'm about to grade?
- Could two experts disagree on what the task is asking? *If yes, revise the prompt.*
- Is there a single clear interpretation, or multiple valid readings? *If multiple valid readings, revise the prompt.*
- Are the hints necessary to solve the problem, or do they just catch existing failures of the model? *If they are intended to catch existing failures they should be excluded.*

The correct answer should be inferable from the task description. A skilled person reading only the prompt should be able to determine what good looks like.

Describe behavior, not keystrokes

Describe the desired behavior and let the model choose the implementation. Don't specify exact line numbers, variable names, or step-by-step keystrokes.

Example 1

- **Do:** "Add input validation to this function. Keep the public API the same and make minimal changes."
- **Over-specified:** "Add exactly three if statements before line 42, use this variable name, don't change anything else."

Example 2

- **Do:** "I want to make sure the password reset flow is safe before modifying it. Trace the reset token lifecycle, from creation through email delivery, token validation, password update, and reuse prevention. Identify any gaps or edge cases."
- **Over-specified:** "Be extremely detailed and inspect all authentication logic line by line."

Sustaining a long conversation

A natural workflow should change based on what happened. If the model found a bug, ask it to isolate

A natural workflow should change based on what happened. If the model found a bug, ask it to isolate the root cause. If it finds missing tests, ask for regression coverage. If it raises a blocker, respond to the blocker.

Make token usage come from real work

A high-token prompt needs a real engineering motive: understanding a subsystem before changing it, reviewing tests before touching code, checking whether a suspected bug is real, or designing a regression test from confirmed behavior.

The tell is focus: one clear concern, not a list of territory. If the prompt would make sense as a message from one engineer to another, it's probably natural. If it mostly reads as an instruction to produce a large volume of text, it isn't.

Strong prompts generate tokens through: codebase traversal, lifecycle tracing, dependency inspection, bug investigation, test review, scoped implementation, regression test design, and failure analysis.

Weak prompts generate tokens through: vague demands for exhaustiveness, unrelated subsystem lists, repeated instructions to continue, requests for maximum detail with no goal, artificial line-by-line review, unsupported edge case generation, and verbosity in place of evidence.

Example 1 (billing)

- **Natural:** "I want to know whether the billing sync can create duplicate invoices. Trace the sync job, invoice lookup logic, external API response handling, and existing tests. Focus on duplicate creation, sync failures, and retry behavior."
- **Unnatural:** "Analyze billing, users, teams, invoices, permissions, notifications, integrations, logs, metrics, frontend state, backend state, database, tests, and every edge case in detail."

Example 2 (cache)

- **Natural:** "I think the cache may not be invalidated after a user changes their organization role. Can you trace the role update and verify whether stale permissions remain active after the change?"
- **Unnatural:** "Analyze the cache, roles, users, organizations, permissions, routes, services, tests, and all related systems in detail. Cite each finding and leave comments."

What to Avoid

Templated Prompting

Structure can be useful, but each turn should *not* feel like the same prompt with different verbiage swapped in. For example, "Investigate [subsystem]. Trace [paths]. Build [matrix]. Classify [risks]. Produce [report]." repeated turn after turn with only the subsystem swapped fails to simulate natural human prompting.

- We are training a model that will respond to prompts from normal users. If we consistently send extremely detailed prompts, with all the context and information, and use a template, the model will learn based on this type of prompt.
- We know that real life is very different from this. Therefore, variability is good and recommended.

Confabulation

Don't invent bugs, fake requirements, or unsupported claims just to give the model something to do. If a problem isn't confirmed by code, logs, tests, or prior output, frame it as something to investigate, and let the model determine whether the suspected issue is real, harmless, or unsupported.

- **Do:** "I'm concerned there may be rollback issues. Trace the relevant paths and tell me whether the concern holds up."
- **Don't:** "The rollback logic is broken. Find the bug and fix it."

Evaluation contamination

Avoid naming the behavior being evaluated. Instructions like "be honest" or "don't hallucinate" feed the model the answer and taint the axis. Tie the behavior to the task instead; the natural version still tests the same behavior without naming it.

- **natural:** "Only call something a bug if code, a failing test, logs, or reproducible behavior supports it."
- **unnatural:** "Be honest, avoid hallucination, and keep it clear."

Prescribing the conversation shape

Recursion and conversation steps

Avoid tells like "for this final turn" or "as a last polish pass." Real engineers rarely know which turn is the last; leaking that structure breaks realism and signals a test.

Stapling two tasks together

We sometimes see two unrelated tasks chained into one to hit a target pass rate. Yes, it's harder. It's also not the kind of problem we're trying to get better at — it's just two easier problems learned the long way around. If a task has multiple parts, the parts need to genuinely build on each other. If you could split them apart cleanly, they were never one task.

Farming a known weakness

If the model reliably fumbles on a topic (e.g., a particular datepicker, a custom tool) it's tempting to build difficulty around that, because the pass rates come out right. But when a big slice of your tasks all bottom out in the same underlying failure, we get a homogeneous training signal. The model is quite promptable; most of the time you should prompt around a common failure rather than lean on it. A few tasks probing a known weakness are fine. It shouldn't be the main reason your tasks are hard. When the model fails a task half the time, we don't want every failure to be identical across all of your submissions

Slight variations

Fifty tasks saying "analyze this dataset" with fifty different CSVs is one task. Vary what the analysis is: trend detection, anomaly finding, forecasting. Quick test: if your whole set fits in one sentence ("they're all X with different inputs"), it isn't diverse enough.

Worked example: an over-broad prompt

The following prompt breaks every principle above:

"Act as a principal security architect joining a Fortune 100 company. Do not design, plan, recommend, or write code yet. Only investigate and model the current state in extreme detail. Attached repos/tools: OWASP LLM Top 10, PyRIT, Garak, Promptfoo, DeepTeam, Semgrep, OWASP ZAP, Dependency-Track, TruffleHog, Nuclei, Amass, reNgine. Recursively inspect all files in these repos. Assume: 200k employees, 50k apps, 10k APIs, 1k AI/LLM systems, 500 AI agents, 100k repos, 15 AWS/Azure/GCP environments, thousands of CI/CD pipelines, global regulatory scope. Produce a deep enterprise discovery report for AppSec, OffSec, AI Security, AI Agent Security, Cloud Security, Supply Chain Security, ASM, governance, risk, and security operations. For every domain, create inventories, taxonomies, stakeholder maps, data-flow maps, trust-boundary maps, ownership maps, dependency maps, threat-surface maps, maturity gaps, unknowns, assumptions, failure modes, and scaling bottlenecks. For each attached repo/tool, analyze role, users, data produced, data consumed, integrations, risks detected, limitations, false positives, false negatives, enterprise constraints, ownership, operational burden, and reporting value. Use tables, matrices, trees, and text diagrams. Please expand every category into subcategories and examples. Be exhaustive. Do not summarize."

Why it fails:

- It demands a huge, unfocused product of work across every security domain, artifact type, tool, and stakeholder map at once. This looks sophisticated but isn't cohesive or focused.
- "Recursively inspect all files in these repos" is a dubious instruction.
- "Be exhaustive" and "do not summarize" are red flags for artificial token burning. Real work of this kind would include summaries, prioritization, and evidence labels.
- "Do not summarize" pushes the model toward endless expansion instead of useful communication.

